@ aman_views

1.

# TESTNG
## in Automation Testing

### 1. What is TestNG ?

- TestNG is a Java testing framework used for unit, integration, UI and API automation.
- It provides annotations, grouping, parallel execution and reporting capabilities.
- TestNG stands for Test Next Generation.

### 2. Why use TestNG ?

- Easy test organization and management.
- Rich set of annotations.
- Supports parallel execution.
- Data-driven testing with parameters.
- Built-in reporting (HTML reports).
- Flexible and configurable (via XML).
- Widely used and well supported by the community.

### 3. TestNG vs JUnit

| Feature | TestNG | JUnit |
|---|---|---|
| Annotations Support | Yes (richer) | Yes (basic) |
| Parallel Execution | Yes | Limited |
| Grouping | Yes | No |
| Data Provider | Yes | No (limited support) |
| XML Suite Support | Yes | No |
| Reporting | Built-in (HTML) | Needs plugins |
| Test Dependency | Yes | Limited |

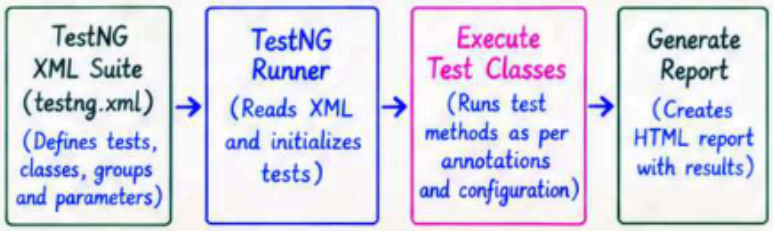
### 4. Key benefits for QA / SDET

- Create reusable and maintainable test suites.
- Execute regression suites efficiently.
- Better test organization using groups and annotations.
- Supports data-driven and parameterized tests.
- Easy integration with Selenium, Playwright and API tests.
- CI/CD friendly with parallel execution and detailed reporting.

### 5. Where it fits in real projects ?

- UI test automation (Web / Mobile).
- API test automation.
- Smoke, regression and end-to-end test suites.
- CI/CD pipelines for automated execution.
- Used in hybrid frameworks with Selenium / Playwright.

### 6. Basic flow of TestNG execution




### 7. Key annotations snapshot

| Annotation | Purpose |
|---|---|
| @Test | Marks a method as a test case. |
| @BeforeSuite | Runs once before all tests. |
| @BeforeClass | Runs before all test methods in a class. |
| @BeforeMethod | Runs before each test method. |
| @AfterMethod | Runs after each test method. |
| @AfterClass | Runs after all test methods in a class. |
| @AfterSuite | Runs once after all tests. |
| @DataProvider | Provides test data to test methods. |

### 8. Key takeaway

★ TestNG is a powerful and flexible framework for structured, maintainable and scalable automation testing.

# 2. TESTNG ANNOTATION LIFECYCLE

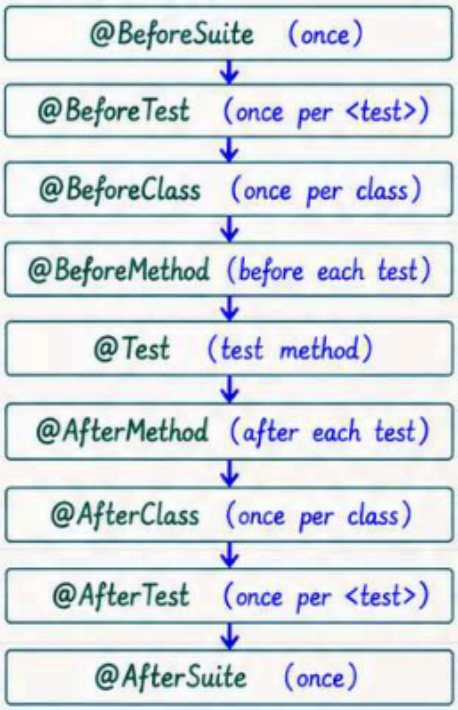
## 1. What are TestNG annotations ?

- TestNG provides annotations to control the setup, execution and cleanup of tests.
- Annotations define when a method should run during the test lifecycle.

## 2. Main lifecycle annotations

| Annotation | Purpose |
|---|---|
| @BeforeSuite | Runs once before all tests in the suite. |
| @BeforeTest | Runs before any test in <test> tag. |
| @BeforeClass | Runs once before all test methods in the class. |
| @BeforeMethod | Runs before each @Test method. |
| @Test | Marks a method as a test case. |
| @AfterMethod | Runs after each @Test method. |
| @AfterClass | Runs once after all test methods in the class. |
| @AfterTest | Runs after all tests in <test> tag. |
| @AfterSuite | Runs once after all tests in the suite. |

## 3. Execution order




## 4. Small code example

```
import org.testng.annotations.*;
public class SampleTest {
    @BeforeClass
    public void setUpClass() {
        System.out.println("BeforeClass - setup");
    }
    @BeforeMethod
    public void setUp() {
        System.out.println("BeforeMethod - login");
    }
    @Test
    public void testExample() {
        System.out.println("Test case execution");
    }
    @AfterMethod
    public void tearDown() {
        System.out.println("AfterMethod - logout");
    }
    @AfterClass
    public void cleanUpClass() {
        System.out.println("AfterClass - cleanup");
} }
```

## 5. When to use which ?

- Use suite-level annotations for global setup (e.g. start reports, DB connection).
- Use class-level annotations for one-time setup per test class.
- Use method-level annotations for actions that need to run before/after each test.
- Use cleanup annotations to release resources.

## 6. Real project usage

- Browser setup in @BeforeClass (open browser and launch application).
- API context setup in @BeforeSuite (base URL, auth token, headers).
- Test data preparation in @BeforeMethod (create test data, reset state).
- Cleanup in @AfterMethod / @AfterClass (close browser, delete test data, release resources).

## 7. Key note

★ BeforeMethod runs before every test method, while BeforeClass runs once per class.

3.

# IMPORTANT TESTNG ANNOTATIONS

## 1. Parameters & Optional

- @Parameters reads values from testng.xml and passes them to the test method.
- @Optional provides a default value if the parameter is not defined in testng.xml.

```
@Test
@Parameters({"browser"})
public void testLogin(@Optional("chrome")
    String browser) {
        System.out.println("Browser: " + browser);
}
```

## 2. DataProvider

- @DataProvider supplies multiple datasets to a test method.
- Useful for running the same test with different inputs.

```
@DataProvider(name = "loginData")
public Object[][] getData() {
    return new Object[][] {
        { "user1", "pass1" },
        { "user2", "pass2" }
    };
}
@Test(dataProvider = "loginData")
public void testLogin(String username, String password) {
    // test logic
}
```

## 3. BeforeGroups / AfterGroups

- @BeforeGroups runs before the execution of a specific group.
- @AfterGroups runs after the execution of a specific group.

```
@BeforeGroups("smoke")
public void setUp() {
    // setup for smoke group
}

@AfterGroups("smoke")
public void tearDown() {
    // cleanup after smoke group
}
```

## 4. Factory

- @Factory creates multiple instances of a test class.
- Useful when tests need to run with different data or configuration.

```
@Factory
public Object[] createInstances() {
    return new Object[] {
        new LoginTest("chrome"),
        new LoginTest("firefox")
    };
}
```

## 5. Listeners

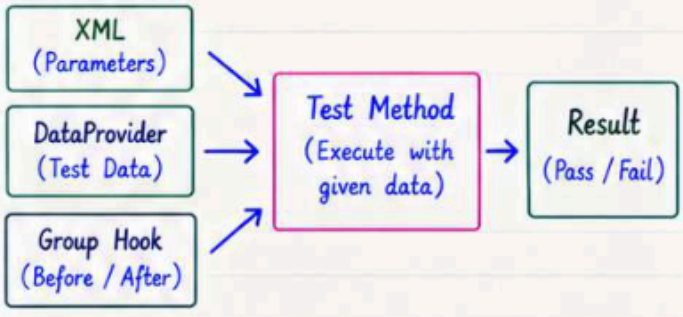
- @Listeners attaches listener classes to test classes.
- Useful for reporting, screenshots, logging and test result tracking.

```
@Listeners({
    CustomListener.class
})
public class LoginTest {
    @Test
    public void test() {
        // test logic
    }
}
```

## 6. Quick use table

| Annotation | Use |
|---|---|
| @Parameters | Reads values from testng.xml |
| @Optional | Provides a default value if parameter not found |
| @DataProvider | Supplies multiple datasets to a test method |
| @BeforeGroups | Runs before a specific group |
| @AfterGroups | Runs after a specific group |
| @Factory | Creates multiple test class instances |
| @Listeners | Attaches listener classes for reporting / logging |

## 7. Mini flow




## 8. Key takeaway

★ Advanced annotations make tests flexible, reusable, and framework-friendly.

# 4. TEST ANNOTATION ATTRIBUTES

## 1. What is @Test ?

- @Test is the core annotation in TestNG to mark a method as a test case.
- It provides various attributes to control the execution, grouping, dependencies and behavior of the test.

## 2. Common attributes

| Attribute | Meaning |
|---|---|
| priority | Defines the execution order (lower value runs first). |
| enabled | Enables or disables the test (true by default). |
| description | Provides a brief description of the test. |
| groups | Assigns the test to one or more groups. |

## 3. Dependency & control attributes

| Attribute | Meaning |
|---|---|
| dependsOnMethods | Specifies methods that must run before this test. |
| dependsOnGroups | Specifies groups that must run before this test. |
| alwaysRun | If true, the test runs even if dependent methods fail. |

## 4. Other useful attributes

- timeOut – Maximum time (in milliseconds) allowed for the test to complete.
- expectedExceptions – Specifies expected exception(s) for the test.
- invocationCount – Number of times to invoke the test method.
- threadPoolSize – Number of threads for parallel invocation.
- dataProvider – Specifies the data provider method for data-driven testing.

## 5. Code example

```
import org.testng.annotations.Test;
public class ApiTest {
    @Test(priority = 1,
          groups = {"smoke"},
          timeOut = 5000,
          dependsOnMethods = "login")
    public void getUserDetails() {
        // test implementation
    }
}
```

## 6. Real project usage

- Use groups (e.g. smoke) to run a subset of critical tests.
- Set timeOut for API tests to avoid hanging tests.
- Disable unstable tests using enabled = false until fixed.
- Use invocationCount for retry or repeated execution (e.g. to check consistency).

## 7. Key note

★ Use @Test attributes to control order, grouping, dependency, and behavior.

5.

# ASSERTIONS IN TESTNG

## 1. What are assertions ?

- Assertions validate the actual result with the expected result.
- They determine whether a test passes or fails.

## 2. Common hard asserts

| Assertion Method | Purpose |
|---|---|
| assertEquals() | Checks if two values are equal. |
| assertTrue() | Checks if a condition is true. |
| assertFalse() | Checks if a condition is false. |
| assertNull() | Checks if a value is null. |
| assertNotNull() | Checks if a value is not null. |
| fail() | Fails the test with a message. |

## 3. SoftAssert

- SoftAssert allows multiple assertions to be executed, even if some fail.
- All failures are reported at the end when we call assertAll().

```
import org.testng.asserts.SoftAssert;

SoftAssert soft = new SoftAssert();
soft.assertEquals(actualTitle, "Expected Title");
soft.assertTrue(isDisplayed);
soft.assertAll(); // reports all failures
```

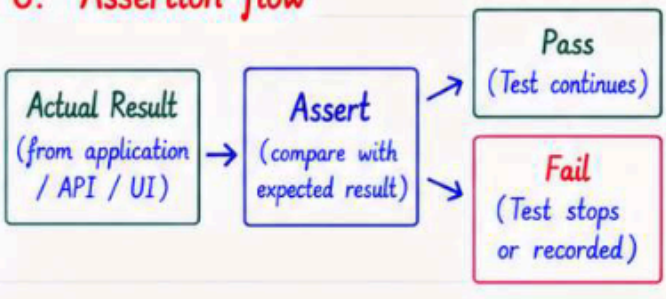
## 4. Hard vs Soft Assert

| Aspect | Hard Assert | Soft Assert |
|---|---|---|
| Behavior | Stops test execution on first failure. | Continues execution and collects all failures. |
| Use Case | Critical validations (where next steps depend on result). | Multiple independent validations in the same test. |
| Reporting | Reports one failure at a time. | Reports all failures after assertAll(). |
| Class | Assert (in TestNG) | SoftAssert (in TestNG) |

## 5. Real validation examples

- Validate page title : assertEquals(driver.getTitle(), "Expected Title").
- Validate API status code : assertEquals(response.getStatusCode(), 200).
- Validate response field : assertEquals(data.get("name"), "John Doe").
- Validate element visibility : assertTrue(element.isDisplayed()).

## 6. Assertion flow




## 7. Key takeaway

★ Assertions verify expected behavior and are the core of every automated test.

6.

# DATA-DRIVEN TESTING

## 1. What is data-driven testing ?

- Run the same test with multiple sets of input data.
- Helps to validate functionality with different data combinations.

## 2. DataProvider syntax

Use @DataProvider to provide test data. It returns a two-dimensional Object array.

```
@DataProvider(name = "loginData")
public Object[][] loginData() {
    return new Object[][] {
        { "user1", "pass1" },
        { "user2", "pass2" }
    };
}
```

## 3. Using DataProvider in @Test

Use the dataProvider name in @Test to run the test with multiple data sets.

```
@Test(dataProvider = "loginData")
public void testLogin(String username,
                      String password) {
   // test logic using username and password
   System.out.println("Login with: " + username);
}
```

## 4. Parameters & Optional

- Apart from DataProvider, TestNG also allows passing values using testng.xml with @Parameters.
- Useful for environment specific values like URL, browser, etc.

```
@Test
@Parameters({ "baseUrl" })
public void testHomePage(String baseUrl) {
   System.out.println("URL: " + baseUrl);
}
```

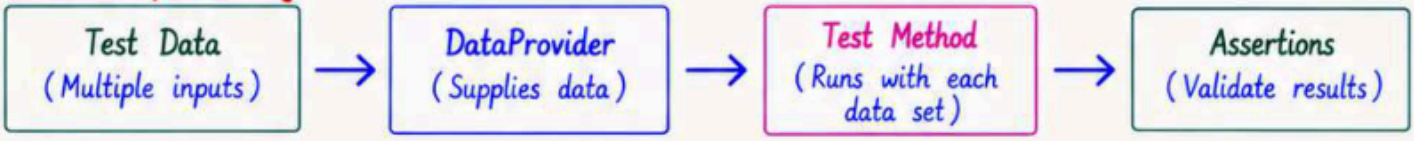
## 5. DataProvider vs Parameters

| Aspect | DataProvider | Parameters |
|---|---|---|
| Source | Provided in code | Provided in testng.xml |
| Use case | Multiple data sets | Environment / config values |
| Data type | Object[][] | Single value (String, etc.) |
| Scope | For test methods | For test methods / classes |
| Best for | Test data combinations | Config and environment setup |

## 6. Common use cases

- Login credentials (valid / invalid).
- Search data (different keywords).
- API payload combinations.
- Browser matrix (Chrome, Firefox, Edge, etc.).

## 7. Mini flow diagram




## 8. Key takeaway

★ Data-driven testing helps run the same test with many datasets efficiently.

7.

# GROUPS, DEPENDENCIES & TEST CONTROL

## 1. What are groups ?

- Groups help to organize and categorize tests based on functionality or purpose.
- Examples: smoke, regression, sanity, integration, ui, api, etc.
- You can include or exclude groups while running tests.

## 2. Include / exclude groups

```
<suite name="TestSuite">
    <groups>
        <run>
            <include name="smoke"/>
            <exclude name="slow"/>
        </run>
    </groups>
</suite>
```

## 3. dependsOnMethods

- A test method can depend on another test method. If the dependent method fails, the current method will be skipped.
- Useful for sequential test flows.

```
@Test
public void loginTest() {
    // login steps
}
@Test(dependsOnMethods = "loginTest")
public void paymentTest() {
    // payment steps
}
```

## 4. dependsOnGroups

- A test method can depend on a group.
- It will run only if all tests in the specified group pass.
- Useful for multi-stage test execution.

```
@Test(groups = "cart")
public void addToCartTest() {
    // add to cart steps
}
@Test(dependsOnGroups = "cart")
public void checkoutTest() {
    // checkout steps
}
```

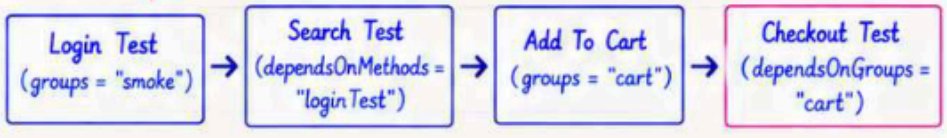
## 5. Other control options

| Option | Description |
|---|---|
| priority | Defines the execution order (lower value runs first). |
| enabled | Enable or disable a test method (true or false). |
| alwaysRun | Run the test even if its dependencies fail. |

## 6. Real project example

- Run smoke group on every build (quick feedback).
- Run regression group at night (full test suite).
- Use dependency-based flows for critical journeys (e.g. login → search → checkout).
- Organize tests using groups like smoke, regression, sanity, api, ui, etc.
- Exclude slow or unstable tests when needed.

## 7. Mini flow




**Smoke Group**
- Login Test
- Search Test

**Regression Group**
- All functional tests (including checkout)
- Run at night

## 8. Key takeaway

★ Groups and dependencies help run the right tests in the right order.

8.

# TESTNG XML & SUITE CONFIGURATION

## 1. What is testng.xml ?

- testng.xml is the suite configuration file used to control the execution of TestNG tests.
- It allows us to define test suites, include/exclude classes, set groups, pass parameters and configure execution behavior.

## 2. Main tags

| Tag | Purpose |
|---|---|
| <suite> | Root element for the test suite. |
| <test> | Groups classes to run as a test. |
| <classes> | Holds multiple <class> tags. |
| <class> | Specifies a test class to run. |
| <groups> | Include or exclude test groups. |
| <parameter> | Passes parameters to tests. |

## 3. Basic XML example

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name="MyTestSuite">
    <test name="AuthenticationTests">
        <classes>
            <class name="tests.LoginTest" />
            <class name="tests.HomePageTest" />
        </classes>
    </test>
</suite>
```

## 4. Groups & parameters

```
// Include a group (smoke)
<test name="SmokeTests">
    <groups>
        <run>
            <include name="smoke" />
        </run>
    </groups>
</test>

// Pass a parameter (browser)
<test name="CrossBrowserTests">
    <parameter name="browser" value="chrome" />
    <classes>
        <class name="tests.BrowserTest" />
    </classes>
</test>
```

## 5. Parallel settings

- parallel="tests" → Run multiple <test> tags in parallel.
- parallel="classes" → Run multiple classes in parallel.
- parallel="methods" → Run test methods in parallel.
- thread-count="N" → Specifies the number of parallel threads.

## 6. Execution hierarchy

**Suite** (testng.xml) — Defines the overall suite configuration
→ **Test** (<test>) — Groups classes and sets parameters, groups, etc.
→ **Class** (<class>) — Specifies the test class to run
→ **Methods** (@Test) — Individual test methods inside the class

## 7. Key note

★ testng.xml gives full control over suites, groups, parameters, and parallel runs.

# 9. LISTENERS, RETRY & REPORTING

## 1. What are listeners ?

- Listeners in TestNG track test events like start, success, failure, skip, etc. and allow us to perform custom actions such as logging, reporting, screenshot capture, etc.

## 2. Common listener interfaces

| Interface | Use |
|---|---|
| ITestListener | Listen to test execution events (test start, success, failure, etc.) |
| ISuiteListener | Listen to suite level events (onStart, onFinish). |
| IInvokedMethodListener | Listen to method invocation events (before and after method). |

## 3. Useful callbacks (ITestListener)

- onTestStart() - Called when a test starts.
- onTestSuccess() - Called when a test passes.
- onTestFailure() - Called when a test fails.
- onStart() - Called when the suite starts.
- onFinish() - Called when the suite finishes.

## 4. Retry analyzer

- Retry analyzer allows us to re-run failed tests automatically based on a condition (e.g. retry 2 times on failure).
- Implement IRetryAnalyzer and return true to retry the test.

```
@Test(retryAnalyzer = Retry.class)
public void testLogin() {
    // test code
}
```

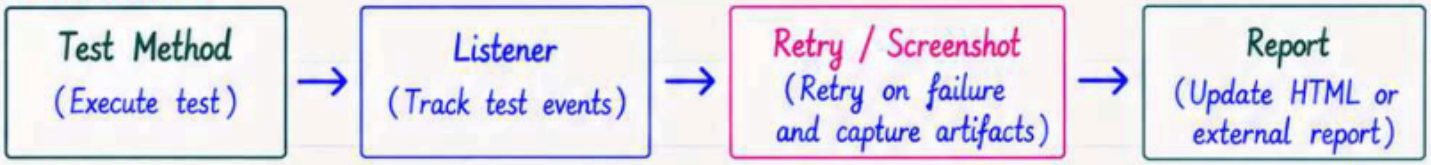
## 5. Failure actions

- Take screenshot on test failure.
- Capture application logs / console logs.
- Attach request / response details (for API tests).
- Update report with failure details.

## 6. Reporting

- TestNG generates default HTML reports (surefire-reports / test-output).
- Emailable report can be generated for sharing results via email.
- Can integrate with external reporting tools like Extent Reports or Allure for better visualization.

## 7. Mini flow




## 8. Key takeaway

★ Listeners and retries make automation more observable and reliable.

# 10. PARALLEL EXECUTION & REAL PROJECT USAGE

## 1. What is parallel execution?

- Parallel execution allows multiple test cases to run at the same time using multiple threads.
- It helps to reduce the overall execution time and speeds up the feedback cycle.

## 2. Parallel modes

| Mode | Meaning |
|---|---|
| tests | Run multiple <test> tags in parallel. |
| classes | Run multiple test classes in parallel. |
| methods | Run multiple test methods in parallel. |

## 3. Thread safety

- Be careful with shared state (e.g. static variables, single WebDriver instance).
- Use ThreadLocal WebDriver for parallel execution.
- Keep test data isolated for each thread to avoid data conflicts.
- Avoid sharing mutable objects across tests.

## 4. Framework structure

```
MyTestNGFramework/
├── src/
│   ├── tests/        (Test classes)
│   ├── pages/        (Page Object classes)
│   ├── utils/        (Utilities, helpers)
│   ├── listeners/    (TestNG listeners)
│   └── data/         (Test data, JSON/Excel)
│   └── config/       (Configurations)
├── testng.xml        (TestNG suite file)
└── pom.xml           (Maven dependencies)
```

## 5. CI / CD usage

- Use Maven Surefire plugin to run TestNG tests.
- Integrate with Jenkins or GitHub Actions.
- Run smoke tests on every pull request (PR).
- Run full regression suite on merge / nightly.
- Publish TestNG HTML reports as build artifacts.

## 6. Real project flow

- Set up TestNG suite with required parallel mode in testng.xml.
- Run smoke tests for each PR to get quick feedback.
- Trigger full regression suite after merge / nightly.
- Review results in TestNG HTML report and take action on failures.

## 7. Mini flow diagram




## 8. Final takeaway

★ TestNG scales from basic tests to enterprise automation frameworks.